passed argument string, and the next invocation of the function will result in a different value based on the earlier call. The state remembered by a function could be a static value or a global value. Stateful functions are bad, because the behaviour of the functions will change depending on whether you have called it before.

Non-reentrant functions

A function is 'reentrant' if it can be safely called again (i.e., it can safely 'reenter' the function) even if it is already part of a call. For example, assume that a function is interrupted by a signal. Now, if the function is entered again from some other control flow, it should be safe to execute that function; now, after this reentered execution completes, the original invocation should be able to complete execution safely.

A function is 'non-reentrant' if it cannot be safely called again if it is already part of a call. Many of the library functions in the original C library are non-reentrant. In general, stateful functions (which we discussed just now) are non-reentrant. So, the standards have defined equivalent C library functions that are reentrant. For example, *strtok_r* is a thread-safe alternative to *strtok*. Note the '*_r*' suffix—it stands for '*reentrant*'. A few examples of reentrant variants of C functions are: *rand_r, asctime_r, ctime_r,* and *localtime_r.*

Functions that do 'too many things'

An example of a function that does too many things at once is *realloc()*. Depending on the argument values passed, it can work like *malloc()* and allocate memory, can work like *free()* and release memory, or reallocate the memory. The method could have perhaps been defined only to do one thing: reallocate memory. To elaborate, here is the declaration of *realloc:*

```
void *realloc(void *ptr, size_t size);
```

Now, depending on the argument values, the behaviour of realloc changes. If *size* < original block, it shrinks the block; if *size* > original block, it expands the block. Now, if *ptr* is *NULL*, it is equivalent to calling *malloc(size);* if *size* is equal to zero and *ptr* is not *NULL*, it is equivalent to calling *free(ptr).* If the *ptr* got moved, then it is equivalent to *free(ptr).* Yes, from this description it is clear that the designer of *realloc* has made it smart — based on the argument values, the behaviour of the function changes. However, preferable to 'smart programming' is 'simple and straightforward programming' in API design, and an important aspect of keeping functions simple is to let each do one thing and do it well.

I have listed some of the mistakes or problems in the C library. The list is not exhaustive but is just meant to give you an idea of the kind of API design problems in the C library. Good designers not only learn from their own mistakes, but also from others' mistakes and from the past. I hope this list has motivated you to improve and become a better designer. END

By: S G Ganesh

The author works for Siemens (Corporate Research & Technologies), Bengaluru. You can reach him at *sgganesh at gmail dot com.*

OSFY Magazine Attractions During 2013-14

MONTH	THEME	FEATURED LIST
March 2013	Virtualisation	Virtualisation Solution Providers
April 2013	Open source Databases	Certification & Training Solution Providers
May 2013	Network Monitoring	Mobile Apps
June 2013	Open Source application development	Cloud
July 2013	Open Source on Windows	Web Hosting Providers
August 2013	Open Source Firewall and Network security	E-mail Service Providers
September 2013	Android Special	Gadgets
October 2013	Kernel Special	IT Consultancy
November 2013	Cloud Special	IT Hardware
December 2013	Linux & Open Source Powered Data Storage	Network Storage
January 2014	Open Source for Web development and deployment	Security
February 2014	Top 10 of Everything on Open Source	IT Infrastructure